Perbandingan Metode Machine Learning Konvensional dan Deep Learning pada Kasus Titanic, Digit Recognizer, dan Disaster Tweets

Nama	**: Muhammad Yoga Isnaeni**
NIM	**: 1202220266**
Kelas	**: SI-46-EDM**
Tanggal Pengumpulan	**: 30 April 2026**
Github Repository	**: https://github.com/yogaisnaeni02/UTS-DEEP-LEARNING.git**

STRATA 1 SISTEM INFORMASI

UNIVERSITAS TELKOM

Semester Genap - Tahun Akademik 2025/2026

BAB I PENDAHULUAN

I.1. Latar Belakang

Kemajuan dalam bidang machine learning dan deep learning telah membuka berbagai pendekatan untuk menyelesaikan masalah klasifikasi pada beragam tipe data, mulai dari data tabular, citra, hingga teks. Metode machine learning konvensional seperti Logistic Regression, Random Forest, Support Vector Machine (SVM), dan Naive Bayes tetap relevan karena sifatnya yang efisien, mudah diinterpretasikan, dan terbukti efektif pada berbagai skenario. Di sisi lain, deep learning memungkinkan model untuk secara otomatis mempelajari representasi fitur yang kompleks melalui lapisan-lapisan pemrosesan, sehingga banyak diterapkan pada data dengan pola yang rumit seperti gambar dan teks (Alzubaidi et al., 2021).

Namun demikian, deep learning tidak selalu merupakan pendekatan terbaik untuk semua jenis dataset. Performanya sangat dipengaruhi oleh jumlah data, kompleksitas pola, kualitas preprocessing, serta kapasitas komputasi yang tersedia. Pada dataset kecil atau data tabular sederhana, metode konvensional seringkali lebih stabil dan kurang rentan terhadap overfitting. Sebaliknya, pada data citra, Convolutional Neural Network (CNN) umumnya lebih efektif karena mampu menangkap pola spasial melalui operasi konvolusi dan pooling (Stanford CS231, 2026). Untuk data teks berurutan, Long Short-Term Memory (LSTM) dirancang untuk menangani ketergantungan jangka panjang antar kata (Hochreiter & Schmidhuber, 1997).

I.2. Tujuan

Laporan ini bertujuan membandingkan kinerja metode konvensional dan deep learning pada tiga kasus dengan karakteristik data berbeda, menganalisis trade-off antara akurasi, waktu training, kebutuhan komputasi, dan interpretabilitas, serta menyusun rekomendasi berbasis hasil eksperimen yang fair dan reprodusibel.

I.3. Ringkasan Pendekatan

Tiga kasus dikerjakan menggunakan dataset dari Kaggle dengan pendekatan sebagai berikut:

1. Tabular - Titanic : Metode konvensional berupa Random Forest dan XGBoost dibandingkan dengan Multi-Layer Perceptron (MLP) berbasis PyTorch. Seluruh eksperimen menggunakan split data yang konsisten dengan random_seed=42.
2. Image - Digit Recognizer/MNIST : Metode konvensional berupa HOG + SVM dibandingkan dengan Deep CNN (arsitektur multi-block dengan Batch Normalization dan Dropout) menggunakan TensorFlow/Keras.
3. Text - Disaster Tweets : Tiga metode konvensional berbasis TF-IDF (Logistic Regression, Linear SVM, Multinomial Naive Bayes) dibandingkan dengan model Embedding + LSTM menggunakan TensorFlow/Keras.

BAB II IMPLEMENTASI

II.1 KASUS 1 : DATA TABULAR: TITANIC

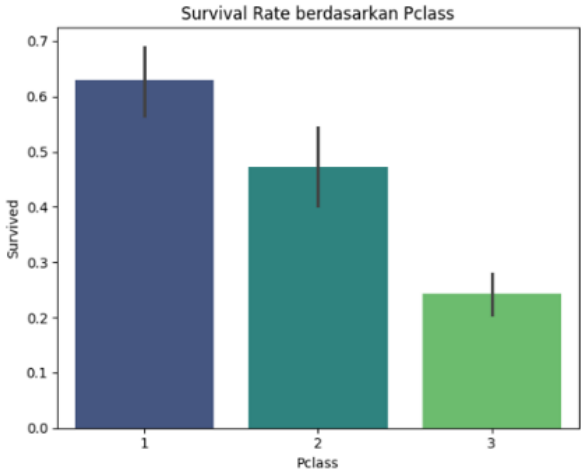
II.1.1 Deskripsi Dataset

Dataset Titanic merupakan dataset klasifikasi biner yang bersumber dari kompetisi Kaggle "Titanic: Machine Learning from Disaster". Target variabel adalah kolom Survived dengan nilai 0 (tidak selamat) dan 1 (selamat). Dataset memiliki karakteristik sebagai berikut:

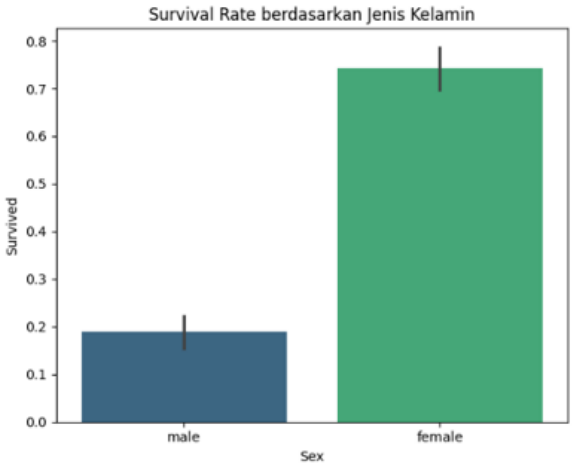
1. Ukuran data training: 891 baris, 11 kolom
2. Ukuran data test: 418 baris, 10 kolom
3. Distribusi kelas: Tidak seimbang penumpang yang tidak selamat lebih banyak dibanding yang selamat
4. Fitur yang tersedia: PassengerId, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked
5. Tantangan utama: Jumlah data yang sangat kecil (891 baris), missing values pada kolom Age (177), Cabin (687), dan Embarked (2), serta kebutuhan feature engineering untuk mengekstrak informasi tambahan dari fitur seperti Name dan Cabin

Kombinasi fitur numerik dan kategorikal menjadikan Titanic sebagai representasi data tabular yang realistis, namun volumenya yang kecil merupakan tantangan utama bagi model deep learning.

II.1.2 Eksplorasi Data (EDA)



Gambar 1: Survival Rate berdasarkan Pclass



Gambar 2 Survival Rate berdasarkan Sex

Eksplorasi data menunjukkan temuan-temuan kunci berikut. Pertama, terdapat ketidakseimbangan kelas yang nyata: penumpang yang tidak selamat (kelas 0) lebih banyak dibanding yang selamat (kelas 1). Kedua, fitur Sex dan Pclass memiliki hubungan yang sangat kuat dengan peluang survival penumpang perempuan dan penumpang kelas satu memiliki tingkat keselamatan yang jauh lebih tinggi. Ketiga, terdapat missing values signifikan terutama pada kolom Cabin (77% kosong), sehingga strategi imputasi dan transformasi menjadi kritis. Keempat, distribusi usia dan tarif (Fare) menunjukkan skewness yang memerlukan penanganan melalui binning atau normalisasi.

II.1.3 Preprocessing

Preprocessing dilakukan secara terstruktur untuk mengubah data mentah menjadi representasi numerik yang dapat diproses oleh semua model. Berikut tahapan preprocessing beserta alasannya:

No	Langkah Preprocessing	Alasan
1	Imputasi Age dengan median berdasarkan Title	Mengatasi 177 missing values; penggunaan median per kelompok lebih representatif daripada median global
2	Imputasi Embarked dengan modus	Mengisi 2 missing values dengan nilai kategorikal paling sering muncul
3	Imputasi Fare dengan median	Mengantisipasi missing value pada data test
4	Ekstraksi Title dari Name	Mengambil informasi sosial/status (Mr, Mrs, Miss, Master, Rare) dari nama penumpang sebagai fitur informatif
5	Ekstraksi Deck dari Cabin	Mengubah informasi kabin yang sebagian besar kosong menjadi fitur Deck dengan nilai 'U' untuk unknown
6	FamilySize = SibSp + Parch + 1	Menggabungkan informasi keluarga menjadi satu fitur numerik Tunggal
7	IsAlone (binary)	Menandai apakah penumpang bepergian sendiri, yang berpengaruh terhadap peluang survival
8	Binning Age dan Fare	Mengelompokkan nilai kontinu agar pola lebih mudah ditangkap model dan membantu mengurangi noise
9	One-hot encoding (Sex, Embarked, Title, Deck)	Mengubah fitur kategorikal menjadi representasi numerik yang dapat diproses model
10	StandardScaler	Normalisasi fitur numerik agar Logistic Regression dan MLP berlatih lebih stabil
11	Train-validation split (80:20, seed=42)	Menjaga konsistensi split antar semua model; 20% data validasi = 179 sampel

Setelah feature engineering selesai, total jumlah fitur yang digunakan adalah 17 fitur.

II.1.4 Metode Konvensional

Dua algoritma konvensional diimplementasikan dengan hyperparameter tuning menggunakan GridSearchCV dengan 5-fold cross-validation:

1. Random Forest dipilih sebagai ensemble method yang mampu menangkap hubungan non-linear antar fitur melalui kombinasi banyak decision tree. Ruang pencarian hyperparameter meliputi n_estimators {100, 200} dan max_depth {5, 10, None}.
2. XGBoost dipilih sebagai gradient boosting method yang dikenal sangat efektif pada data tabular, terutama dengan ukuran kecil-menengah. Ruang pencarian meliputi n_estimators {100, 200}, learning_rate {0.01, 0.1}, dan max_depth {3, 5}.

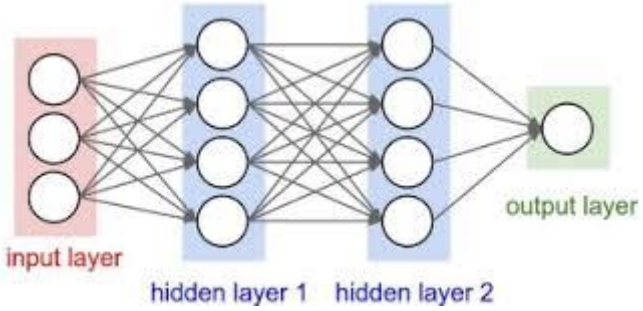
No	Model	Accuracy	Precision	Recall	F1-score
1.	Random Forest	0.8324	0.83	0.82	0.83

2.	XGBoost	0.8212	0.82	0.81	0.81

Random Forest menjadi model konvensional terbaik dengan akurasi 83.24%, menunjukkan bahwa ensemble method mampu menangkap pola non-linear dalam data tabular yang sudah di-engineer dengan baik.

II.1.5 Metode Deep Learning

Arsitektur yang dipilih: Multi-Layer Perceptron (MLP) dengan PyTorch



Gambar 3 Diagram Arsitektur Multi-Layer Perceptron

Arsitektur MLP dipilih karena dataset Titanic merupakan data tabular yang setelah preprocessing berubah menjadi kumpulan fitur numerik terstruktur. Tidak ada struktur spasial (seperti gambar) atau sekuensial (seperti teks) yang memerlukan arsitektur khusus seperti CNN atau LSTM. Dense layer pada MLP mampu mempelajari kombinasi non-linear antar 17 fitur yang tersedia.

Arsitektur yang diimplementasikan:

1. Input layer: 17 fitur
2. Hidden layer 1: Linear(17 → 64) + ReLU + Dropout(0.4)
3. Hidden layer 2: Linear(64 → 32) + ReLU + Dropout(0.3)
4. Output layer: Linear(32 → 1) + Sigmoid (untuk klasifikasi biner)

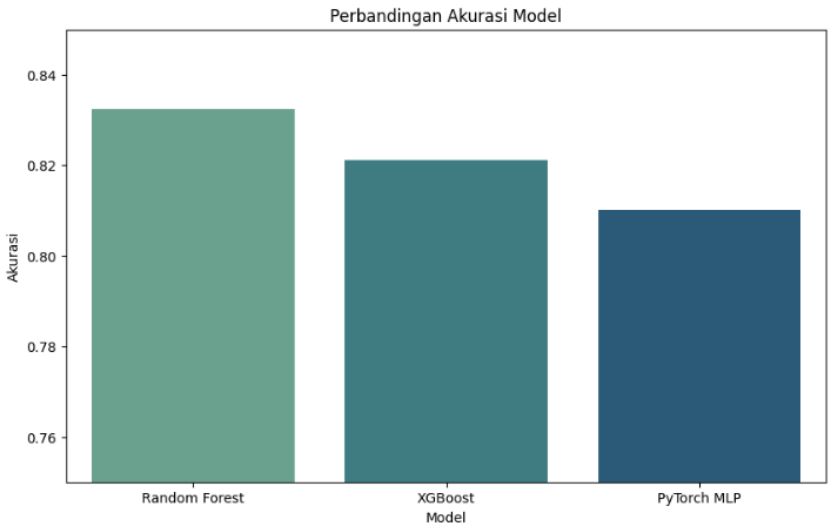
Regularisasi yang diterapkan meliputi Dropout dengan rate tinggi (0.4 dan 0.3) untuk mencegah overfitting pada dataset kecil, weight decay (L2 regularization) melalui parameter optimizer Adam, serta EarlyStopping yang berhenti pada epoch ke-33 untuk mencegah overfitting lebih lanjut.

Hyperparameter training: learning rate = 0.005, batch size = 32, maksimum epoch = 100, optimizer = Adam dengan weight decay.

No	Model	Accuracy	Precision	Recall	F1-score
1.	PyTorch MLP	0.8101	0.81	0.81	0.81

II.1.6 Perbandingan Hasil

No	Model	Jenis Metode	Accuracy	F1-score
1.	Random Forest	Konvensional	0.8324	0.83
2.	XGBoost	Konvensional	0.8212	0.81
3.	PyTorch MLP	Deep Learning	0.8101	0.81



Gambar 4 Perbandingan Akurasi Model

Random Forest menjadi model terbaik pada Kasus 1. MLP menghasilkan akurasi terendah meskipun menggunakan arsitektur dengan regularisasi. Dari sisi waktu training, metode konvensional jauh lebih efisien GridSearchCV untuk Random Forest dan XGBoost dapat selesai dalam hitungan detik hingga menit, sementara MLP memerlukan iterasi epoch yang lebih panjang dengan early stopping di epoch 33.

Pada dataset Titanic (891 baris training), metode konvensional terbukti lebih efektif. Hal ini konsisten dengan prinsip umum bahwa deep learning membutuhkan volume data yang besar untuk mengoptimalkan jutaan parameter. Dengan hanya 712 data training (80% dari 891), MLP tidak memiliki cukup sampel untuk mempelajari pola yang lebih baik dari Random Forest. Selain itu, fitur-fitur yang sudah di-engineer secara manual telah merepresentasikan informasi penting dengan baik, sehingga deep learning tidak mendapat keuntungan dari kemampuan automatic feature learning-nya.

II.1.7 Analisis Error

Error analysis dilakukan pada prediksi XGBoost sebagai representasi metode konvensional terbaik. Ditemukan 32 kesalahan prediksi dari 179 data validasi.

No	Karakteristik Data	Label Asli	Prediksi	Interpretasi
1	Pclass=1, Sex=male, Age=42, Fare=52.55	1 (Selamat)	0 (Tidak Selamat)	Model terlalu mengandalkan pola bahwa penumpang laki-laki dewasa cenderung tidak selamat, meskipun ia dari kelas 1
2	Pclass=1, Sex=male, Age=34, Fare=26.55	1 (Selamat)	0 (Tidak Selamat)	Pola yang sama laki-laki kelas 1 dengan usia produktif tidak selalu mengikuti pola mayoritas
3	Pclass=2, Sex=male, Age=31, Fare=13.00	1 (Selamat)	0 (Tidak Selamat)	Penumpang kelas 2 yang selamat tidak teridentifikasi karena mayoritas laki-laki dari kelas menengah tidak selamat

Pola kesalahan yang dominan adalah false negative pada penumpang laki-laki yang ternyata selamat model terlalu kuat menginternalisasi bias bahwa laki-laki dewasa memiliki peluang survival rendah. Ini

mencerminkan batasan inheren dari data historis yang memang menunjukkan korban didominasi oleh laki-laki, sehingga pengecualian sulit dideteksi tanpa fitur tambahan seperti posisi di kapal atau proximity ke sekoci.

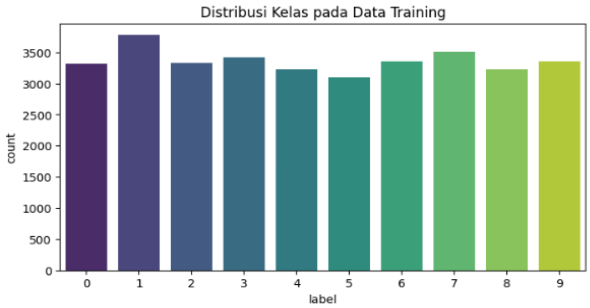
II.2 KASUS 2 DATA IMAGE: DIGIT RECOGNIZER (MNIST)

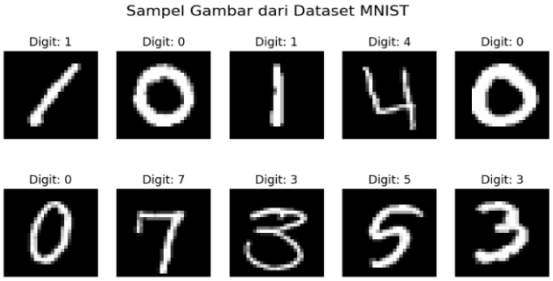
II.2.1 Deskripsi Dataset

Dataset Digit Recognizer berasal dari kompetisi Kaggle yang menggunakan dataset MNIST (Modified National Institute of Standards and Technology). Dataset ini merupakan benchmark klasik untuk klasifikasi citra.

1. Ukuran data training: 42.000 baris × 785 kolom (1 label + 784 piksel)
2. Ukuran data test: 28.000 baris × 784 kolom
3. Jumlah kelas: 10 kelas (digit 0–9)
4. Ukuran gambar: 28×28 piksel, grayscale
5. Distribusi kelas: Relatif seimbang antar 10 kelas digit
6. Split yang digunakan: 80% training (33.600), 20% validasi (8.400), seed=42
7. Tantangan utama: Variasi bentuk tulisan tangan antar individu, serta kemiripan visual antar digit tertentu (3 vs 5, 4 vs 9, 1 vs 7)

II.2.2 Eksplorasi Data (EDA)





Eksplorasi data menunjukkan bahwa distribusi label antar 10 kelas digit relatif seimbang, dengan masing-masing kelas memiliki sekitar 4.000 sampel. Visualisasi contoh gambar memperlihatkan variasi signifikan dalam gaya penulisan antar individu beberapa digit ditulis tebal, tipis, miring, atau dengan proporsi yang tidak standar. Beberapa pasang digit secara visual sangat mirip (misalnya 3 dan 5, atau 4 dan 9), yang menjadi sumber utama kesalahan klasifikasi pada kedua pendekatan.

II.2.3 Preprocessing

No	Langkah Preprocessing	Alasan
1	Normalisasi piksel ÷ 255.0	Mengubah nilai piksel dari rentang [0, 255] ke [0, 1] agar proses training lebih stabil dan konvergen lebih cepat
2	Reshape ke (28, 28) → flatten ke (784,)	Menyesuaikan format input untuk ekstraksi HOG dan SVM (butuh vektor 1D)
3	Reshape ke (-1, 28, 28, 1) untuk CNN	Mempertahankan struktur spasial 2D yang diperlukan oleh lapisan konvolusi
4	Ekstraksi HOG (orientations=9, pixels_per_cell=(7,7), cells_per_block=(2,2))	Mengekstrak fitur bentuk, arah gradien, dan tepi dari gambar 28×28; mempartisi gambar menjadi 16 sel (4×4)

5	Train-validation split (80:20, seed=42)	Konsistensi split antar semua model untuk perbandingan yang fair

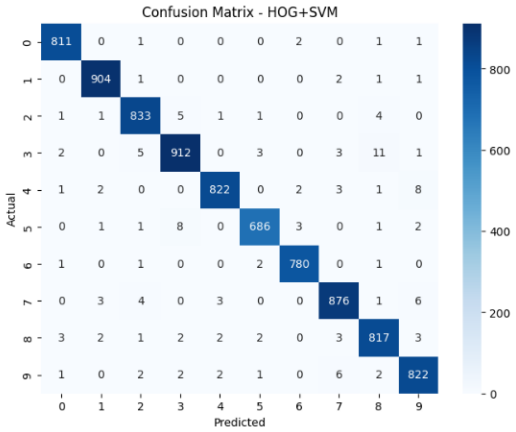
II.2.4 Metode Konvensional

HOG + SVM diimplementasikan sebagai pipeline utama. Histogram of Oriented Gradients (HOG) mengekstrak fitur deskriptor yang merepresentasikan distribusi arah gradien pada setiap region gambar. Fitur HOG kemudian dimasukkan ke Support Vector Machine (SVC) dengan kernel RBF untuk klasifikasi multi-kelas.

Parameter HOG yang digunakan: orientations = 9, pixels_per_cell = (7, 7), cells_per_block = (2, 2). Pilihan pixels_per_cell = (7, 7) membagi gambar 28×28 menjadi 16 sel (4×4 grid), menghasilkan vektor fitur yang representatif tanpa terlalu panjang.

HOG dipilih karena merupakan metode standar untuk ekstraksi fitur visual yang terbukti efektif untuk pengenalan pola bentuk. HOG mampu menangkap karakteristik struktural digit (orientasi garis, lengkungan, sudut) yang tidak bergantung pada variasi pencahayaan atau warna.

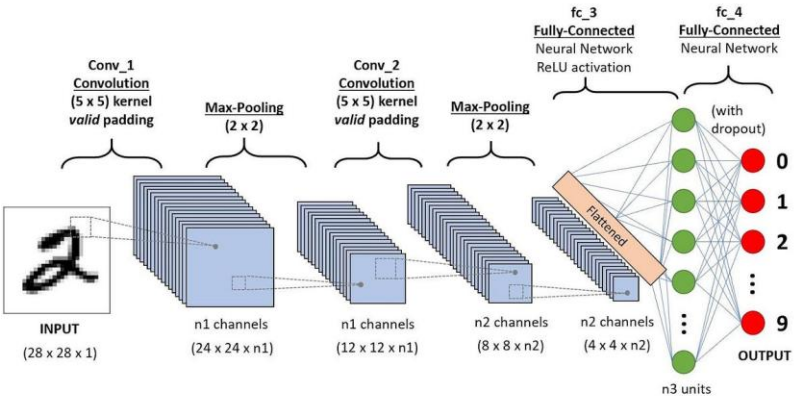
No	Model	Accuracy	F1-score	Misclassification Count
1	HOG + SVM	0.9837	0.98	138 dari 8.400



Gambar 7 Confusion Matrix HOG+SVM

II.2.5 Metode Deep Learning

Arsitektur yang dipilih: Deep CNN dengan TensorFlow/Keras



Gambar 8 Arsitektur CNN

CNN dipilih karena data berupa gambar memiliki struktur spasial yang sangat penting untuk dipertahankan. Lapisan konvolusi mampu menangkap pola lokal seperti garis, tepi, lengkungan, dan sudut yang kritis untuk membedakan digit tulisan tangan. MaxPooling mereduksi dimensi sambil mempertahankan fitur paling dominan, sedangkan Batch Normalization menstabilkan proses training.

Arsitektur Deep CNN yang diimplementasikan (multi-block):

1. Conv Block 1: Conv2D(32, 3×3) → BatchNorm → Conv2D(32, 3×3) → BatchNorm → MaxPooling(2×2) → Dropout(0.25)
2. Conv Block 2: Conv2D(64, 3×3) → BatchNorm → Conv2D(64, 3×3) → BatchNorm → MaxPooling(2×2) → Dropout(0.25)
3. Fully Connected: Flatten → Dense(256) → BatchNorm → Dropout(0.5) → Dense(10, softmax)

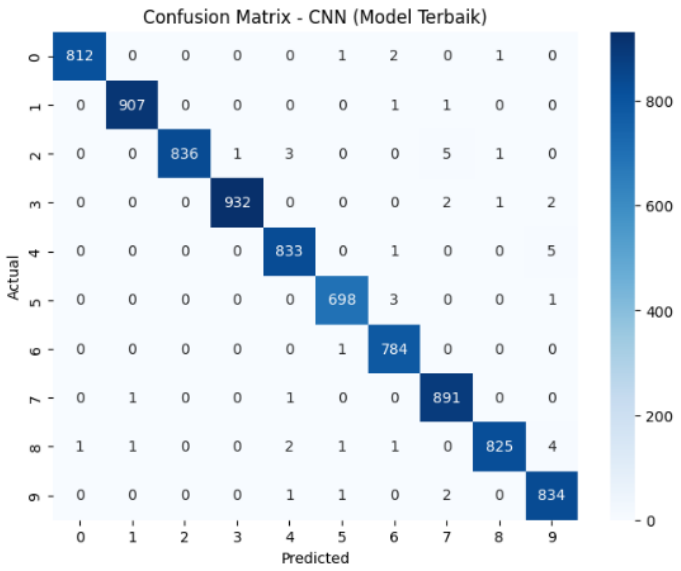
Training dilakukan selama 20 epoch dengan optimizer Adam, categorical crossentropy loss, dan pemantauan val_accuracy. Model mencapai val_accuracy tertinggi pada epoch ke-6 ke atas dengan nilai stabil di atas 0.99.

No	Model	Accuracy	F1-score	Misclassification Count
1	Deep CNN	0.9943	0.99	48 dari 8.400

II.2.6 Perbandingan Hasil

No	Model	Accuracy	Jenis Metode	Misclassification Count
1	Deep CNN	0.9943	Deep Learning	48
2	HOG + SVM	0.9837	Konvensional	138

SVM inference time yang sangat cepat karena data validasi sudah diproses HOG sebelumnya; waktu di atas merupakan waktu prediksi saja.

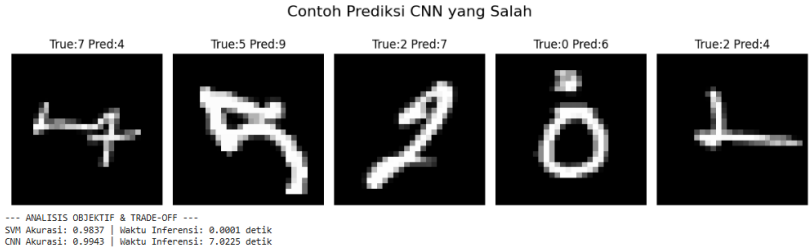


Gambar 9 Confusion Matrix – CNN

CNN memberikan peningkatan yang signifikan: dari 138 kesalahan (HOG+SVM) menjadi hanya 48 kesalahan (CNN), pengurangan kesalahan sebesar 65.2%. Peningkatan akurasi dari 98.37% menjadi 99.43% terlihat kecil secara numerik, tetapi sangat bermakna secara praktis pada skala besar. Trade-off utama adalah waktu training CNN yang jauh lebih lama dibandingkan HOG+SVM.

Perbandingan pola misclassification: Baik HOG+SVM maupun CNN mengalami kesalahan terbanyak pada pasangan digit yang secara visual mirip. Namun, CNN berhasil mengurangi kesalahan pada pasangan seperti (3,5) dan (4,9) karena kemampuannya menangkap fitur spasial hierarkis yang lebih kaya dibanding HOG yang menggunakan deskriptor fixed-size. HOG cenderung gagal ketika tulisan tangan memiliki bentuk yang tidak standar karena deskriptornya tidak adaptif.

II.2.7 Analisis Error



Gambar 10 Lima Sampel Error CNN

Analisis error pada CNN (48 kesalahan dari 8.400 sampel) :

No	Label Asli	Prediksi	Interpretasi
1	7	4	Digit 7 yang ditulis dengan loop tertutup di bagian atas menyerupai angka 4 secara visual
2	5	9	Pola lengkungan ganda pada angka 5 dapat menyerupai angka 9 ketika ditulis dengan proporsi tidak standar
3	2	7	Angka 2 yang ditulis tanpa garis horizontal atau dengan kemiringan ekstrem menyerupai angka 7

Meskipun CNN memiliki performa terbaik, model tetap dapat melakukan kesalahan ketika bentuk digit sangat tidak konvensional, terlalu tipis, terlalu miring, atau memiliki kemiripan visual tinggi dengan kelas lain. Ini menunjukkan bahwa bahkan CNN pun masih terbatas oleh variabilitas tulisan tangan manusia yang ekstrem.

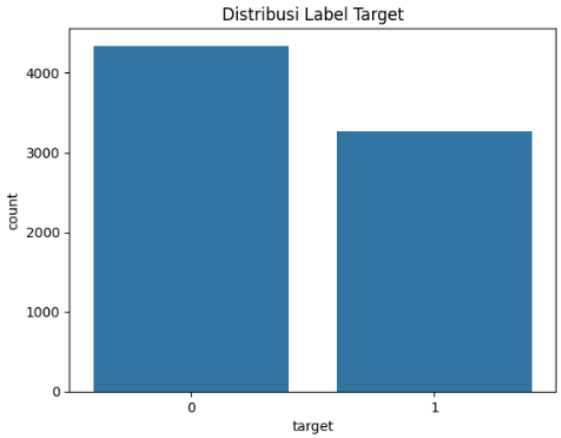
II.3 KASUS 3 DATA TEXT: DISASTER TWEETS

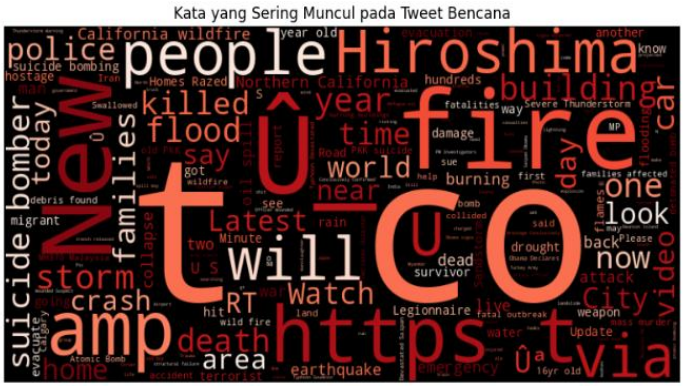
II.3.1. Deskripsi Dataset

Dataset Natural Language Processing with Disaster Tweets berasal dari kompetisi Kaggle "NLP Getting Started". Dataset ini bertujuan mengklasifikasikan apakah sebuah tweet membahas bencana nyata atau tidak.

1. Ukuran data training: 7.613 baris × 5 kolom (id, keyword, location, text, target)
2. Ukuran data test: 3.263 baris
3. Distribusi kelas: Tidak seimbang 4.342 non-disaster (57.0%) dan 3.271 disaster (43.0%)
4. Split yang digunakan: 80% training (6.090), 20% validasi (1.523), stratified, seed=42
5. Fitur utama: Kolom text (teks tweet mentah)
6. Tantangan utama: Teks pendek dan informal, penggunaan bahasa ambigu, banyak URL/mention/hashtag, kata-kata yang bisa bermakna literal maupun metaforis (contoh: "fire", "crash", "storm", "burning")

II.3.2. Eksplorasi Data (EDA)





Eksplorasi data mengungkap beberapa karakteristik penting. Pertama, distribusi kelas menunjukkan ketidakseimbangan sedang (57% vs 43%), yang perlu diperhatikan dalam pemilihan metrik evaluasi F1-score lebih representatif daripada accuracy murni. Kedua, banyak tweet mengandung kata-kata yang secara denotasi berhubungan dengan bencana (fire, crash, storm) namun digunakan dalam konteks non-bencana (metafora, sarkasme, atau percakapan biasa). Ketiga, noise dalam bentuk URL, mention (@username), dan hashtag sangat dominan dalam teks mentah dan perlu dibersihkan.

Contoh dari data:

1. "i cant drown my demons they know how to swim" Label: 1 (Disaster), namun ini adalah ungkapan metaforis
2. "rip the world its burning" Label: 0 (Non-disaster), namun mengandung kata yang berkonotasi bencana

II.3.3. Preprocessing

No	Langkah Preprocessing	Alasan
1	Lowercase	Menyamakan format huruf sehingga "Fire" dan "fire" diperlakukan sebagai token yang sama
2	Hapus URL (http/https/www)	URL tidak memberikan informasi semantik langsung terhadap kelas
3	Hapus mention (@username)	Username bersifat noise dan tidak relevan untuk klasifikasi konten
4	Hapus simbol hashtag (#)	Simbol # dihapus tetapi kata di belakangnya dipertahankan sebagai token bermakna
5	Hapus karakter non-huruf	Mengurangi noise dari angka, tanda baca, symbol
6	Hapus spasi berlebih	Merapikan output teks setelah penghapusan
7	TF-IDF Vectorizer	Representasi numerik teks berdasarkan bobot kemunculan kata (Term Frequency × Inverse Document Frequency)
8	Tokenizer + Padding (max_len=50)	Mengubah teks menjadi sequence angka untuk input LSTM; padding menyamakan panjang semua sequence

II.3.4. Metode Konvensional

Tiga kombinasi TF-IDF + classifier diimplementasikan dengan tuning hyperparameter:

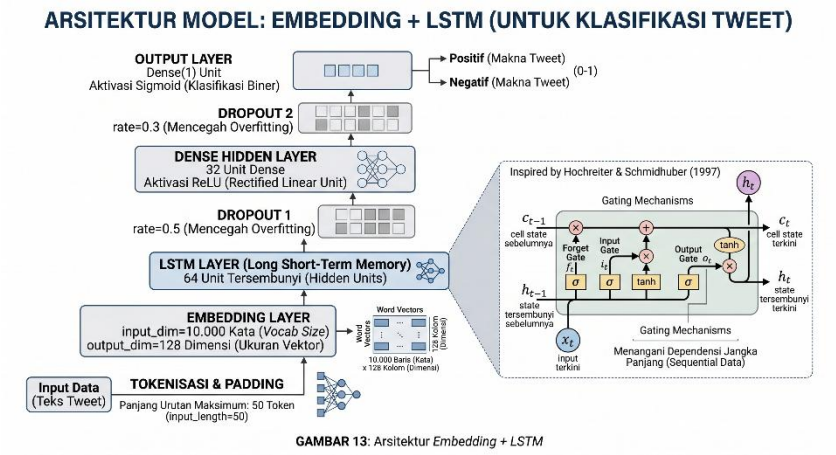
1. TF-IDF + Logistic Regression: TF-IDF unigram (max_features = 10.000) dikombinasikan dengan Logistic Regression. Tuning nilai C{0.1, 1, 10} menunjukkan C=1 memberikan F1-score terbaik (0.7723).
2. TF-IDF + Linear SVM**:** TF-IDF unigram-bigram (max_features=15.000) dikombinasikan dengan LinearSVC. Tuning nilai C{0.1, 1, 10} menunjukkan C=0.1 memberikan F1-score terbaik (0.7637).
3. TF-IDF + Multinomial Naive Bayes: TF-IDF unigram-bigram dikombinasikan dengan MultinomialNB. Tuning nilai alpha{0.1, 0.5, 1.0} menunjukkan alpha=0.5 memberikan F1-score terbaik (0.76).

No	Model	Accuracy	Precision	Recall	F1-score	Training Time
1	TF-IDF + LR	0.8188	0.8387	0.7156	0.7723	0.187 s
2	TF-IDF + LinearSVM	0.7978	0.7764	0.7431	0.7594	0.048 s
3	TF-IDF + Multinomial NB	0.8188	0.8810	0.6682	0.7600	0.005 s

Logistic Regression dengan TF-IDF unigram menjadi model konvensional terbaik. Multinomial NB memiliki precision tertinggi tetapi recall rendah, menandakan model ini lebih konservatif dalam memprediksi kelas disaster.

II.3.5. Metode Deep Learning

Arsitektur yang dipilih: Embedding + LSTM



GAMBAR 13: Arsitektur *Embedding + LSTM*

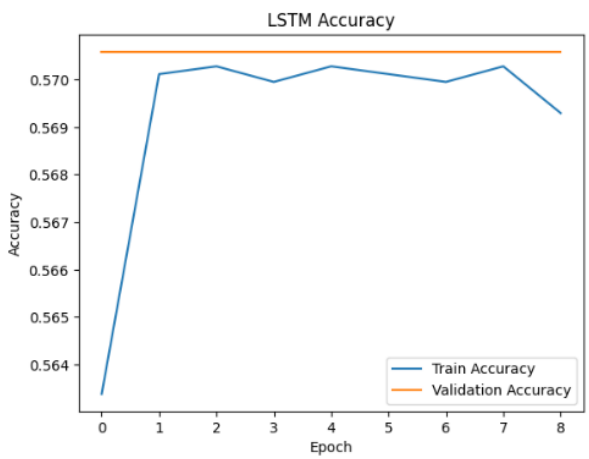
LSTM dipilih karena data teks memiliki bentuk sekuensial urutan kata memberikan informasi penting dalam menentukan makna sebuah tweet. LSTM dirancang untuk menangani dependensi jangka panjang pada data berurutan melalui mekanisme cell state dan gating (Hochreiter & Schmidhuber, 1997). Embedding layer digunakan untuk mengubah token menjadi representasi vektor berdimensi 128.

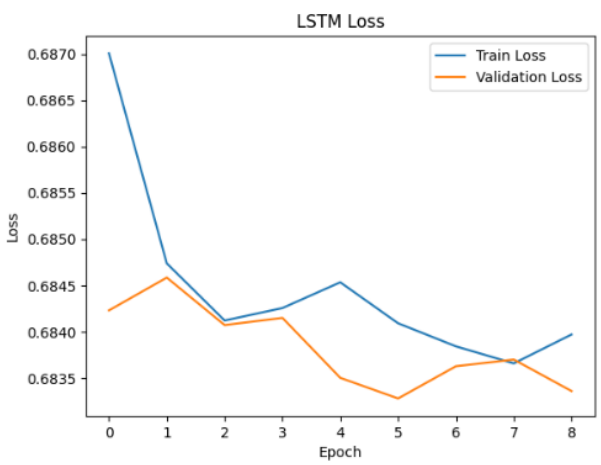
Arsitektur yang diimplementasikan:

1. Embedding layer: input_dim=10.000 vocab, output_dim=128, input_length=50
2. LSTM layer: 64 units
3. Dropout: rate=0.5
4. Dense hidden: 32 units, aktivasi ReLU
5. Dropout: rate=0.3
6. Output layer: Dense(1), aktivasi Sigmoid (klasifikasi biner)

Training dilakukan hingga 10 epoch dengan EarlyStopping (patience=3, monitor='val_loss') dan optimizer Adam. Model berhenti setelah 4 epoch karena val_loss tidak membaik.

]



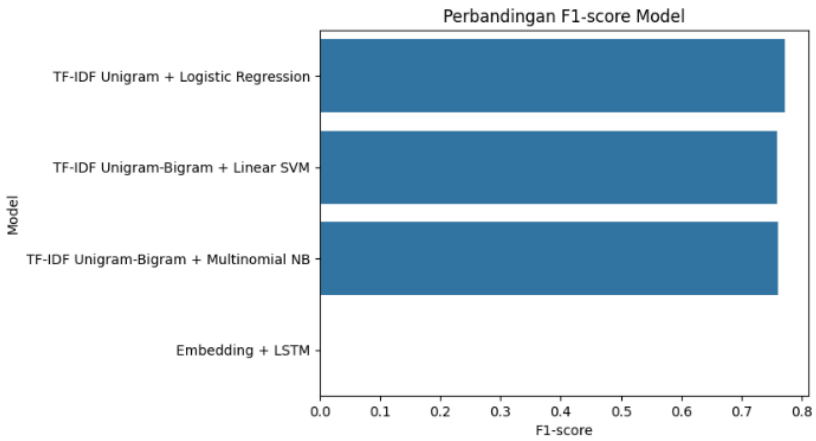


Model	Accuracy	Precision	Recall	F1-score	Training Time
Embedding + LSTM	0.5706	0.0000	0.0000	0.0000	21.67 s

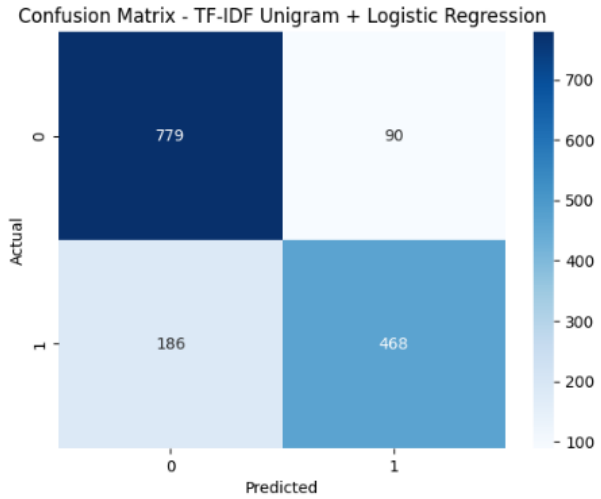
LSTM menghasilkan F1-score 0.0000 karena model secara konsisten memprediksi seluruh data validasi sebagai kelas 0 (non-disaster). Hal ini terlihat jelas dari classification report yang menunjukkan precision=0, recall=0, F1=0 untuk kelas 1, dan accuracy yang hanya setara dengan proporsi kelas mayoritas (57.06%).

II.3.6. Perbandingan Hasil

Model	Jenis Metode	Accuracy	F1-score	Training Time
TF-IDF + LR	Konvensional	0.8188	0.7723	0.187 s
TF-IDF + NB	Konvensional	0.8188	0.7600	0.005 s
TF-IDF + LinearSVM	Konvensional	0.7978	0.7594	0.048 s
Embedding + LSTM	Deep Learning	0.5706	0.0000	21.67 s



Gambar 16 Perbandingan F1-Score Semua Model



Gambar 17 Confusion Matrix TF-IDF + Logistic Regression

Kesenjangan performa antara LSTM dan model konvensional sangat besar. TF-IDF + Logistic Regression mengungguli LSTM dengan selisih F1-score 0.7723 vs 0.0000. Penyebab kegagalan LSTM dianalisis sebagai berikut: (1) LSTM membutuhkan data yang lebih besar untuk mempelajari representasi embedding dari awal (scratch); dengan hanya ~6.000 data training dan panjang tweet yang

pendek (rata-rata <50 token setelah cleaning), embedding tidak dapat dioptimalkan dengan baik; (2) distribusi kelas yang tidak seimbang (57:43) tanpa class weighting menyebabkan LSTM cenderung memprediksi kelas mayoritas; (3) LSTM tanpa pretrained embedding tidak dapat memanfaatkan pengetahuan semantik dari luar dataset.

II.3.7. Analisis Error

Analisis error dilakukan pada tiga model: Logistic Regression, LinearSVM, dan LSTM. Contoh kesalahan Logistic Regression :

No	Tweet	Label Asli	Prediksi	Interpretasi
1	"hollywood movie about trapped miners released in chile..."	1	0	Meskipun mengandung kata "trapped miners", konteks film membuat model tidak yakin → false negative
2	“thu aug gmt utc millcityio theramin sirens"	1	0	Tweet ini sangat pendek dan tidak mengandung kata kunci bencana yang jelas → sulit diklasifikasikan

	text	actual	predicted
5	hollywood movie about trapped miners released ...	1	0
6	thu aug gmt utc millcityio theramin sirens	1	0
7	i cant drown my demons they know how to swim	1	0
13	we need these plants in the pacific during the...	1	0
14	if firefighters acted like cops theyd drive ar...	0	1
16	it looks like a mudslide poor thing greatbriti...	1	0
22	on the flip side im at walmart and there is a ...	1	0
24	maybe we should call israel and tell them were...	0	1
27	i agree with certain cultural appropriation th...	1	0
29	the dress memes have officially exploded on th...	1	0

Gambar 18 Kesalahan Logistic Regression

No	Tweet	Label Asli	Prediksi	Interpretasi
1	"step one get that mass murderers portrait off the yuan"	0	1	Kata "murderers" memicu model memprediksi disaster meskipun konteksnya politis/non-bencana
2	"rip the world its burning"	0	1	Kata "burning" dan "rip" memberikan sinyal bencana yang kuat meskipun ini ekspresi figurative
3	"i cant drown my demons they know how to swim"	1	0	Sama dengan LR metafora sulit ditangkap TF-IDF

	text	actual	predicted
0	step one get that mass murderers portrait off ...	0	1
5	hollywood movie about trapped miners released ...	1	0
7	i cant drown my demons they know how to swim	1	0
10	rip the world its burning	0	1
14	if firefighters acted like cops theyd drive ar...	0	1
20	ignition knock detonation sensorsenso standard ks	0	1
22	on the flip side im at walmart and there is a ...	1	0
24	maybe we should call israel and tell them were...	0	1
27	i agree with certain cultural appropriation th...	1	0
29	the dress memes have officially exploded on th...	1	0

Gambar 19 Kesalahan LinearSVM

LSTM: Karena LSTM memprediksi seluruh sampel sebagai kelas 0, semua tweet berlabel 1 (654 sampel) merupakan false negative, sehingga analisis per-tweet tidak representatif.

Apakah deep learning berhasil menangani kasus sarkasme/konteks? Pada eksperimen ini, tidak. LSTM gagal total. Bahkan model konvensional pun masih mengalami kesulitan pada kasus metafora dan sarkasme, yang memang merupakan batasan representasi berbasis bag-of-words. Untuk mengatasi hal ini diperlukan pendekatan seperti DistilBERT atau pretrained language model yang telah memahami konteks kalimat secara lebih mendalam.

BAB III DISKUSI LINTAS KASUS

III.1 Pada Tipe Data Apa Deep Learning Memberikan Peningkatan Signifikan?

Deep learning memberikan peningkatan yang jelas dan signifikan pada data citra (Kasus 2). CNN berhasil mengurangi jumlah kesalahan klasifikasi dari 138 (HOG+SVM) menjadi 48, dengan peningkatan akurasi dari 98.37% menjadi 99.43%. Keunggulan CNN pada data citra berasal dari kemampuannya mempertahankan dan mengeksploitasi struktur spasial gambar melalui operasi konvolusi yang bersifat translation-invariant. Berbeda dengan HOG yang menggunakan deskriptor fixed-size dan manual, CNN secara otomatis mempelajari hierarki fitur dari tepi dan garis sederhana di lapisan awal, hingga bentuk digit yang kompleks di lapisan dalam. Kedalaman arsitektur (multi-block dengan BatchNorm dan Dropout) juga membantu model generalisasi dengan baik tanpa overfitting.

Ini sesuai dengan literatur yang menyatakan bahwa CNN merupakan arsitektur yang paling sesuai untuk computer vision tasks karena desainnya memang terinspirasi dari mekanisme pemrosesan visual pada makhluk hidup (Alzubaidi et al., 2021; Stanford CS231, 2026).

III.2 Pada Tipe Data Apa Metode Konvensional Masih Kompetitif?

Pada data tabular kecil (Kasus 1) dan data teks pendek (Kasus 3), metode konvensional terbukti lebih unggul.

Pada Titanic, Random Forest mencapai akurasi 83.24% mengalahkan MLP (81.01%). Dengan hanya 712 data training, MLP tidak memiliki cukup sampel untuk mengoptimalkan parameternya secara efektif, meskipun telah dilengkapi regularisasi ketat. Sebaliknya, Random Forest sebagai ensemble method lebih robust terhadap overfitting karena secara inheren melakukan regularisasi melalui bagging.

Fitur hasil feature engineering yang sudah informatif juga berarti bahwa kemampuan automatic feature learning dari deep learning tidak memberikan keuntungan tambahan.

Pada Disaster Tweets, TF-IDF + Logistic Regression mencapai F1-score 0.7723 sementara LSTM gagal total dengan F1=0.0. Tweet yang pendek dan noisy, combined dengan ukuran dataset yang relatif terbatas (~6.000 training), tidak memberikan kondisi yang kondusif untuk LSTM. TF-IDF secara efektif menangkap kata-kata diskriminatif tanpa memerlukan pembelajaran representasi dari awal.

III.3 Trade-off: Akurasi vs Biaya Komputasi vs Interpretabilitas

Aspek	Metode Konvensional	Deep Learning
Akurasi	Unggul pada tabular kecil & teks pendek	Unggul signifikan pada data citra
Waktu Training	Sangat cepat (0.005 s – beberapa menit)	Jauh lebih lama (20+ menit untuk CNN, meski lebih pendek untuk LSTM)
Waktu Inferensi	Sangat cepat (<0.01 s untuk mayoritas)	Lebih lambat (7 detik untuk inferensi 8.400 sampel CNN)
Interpretabilitas	Tinggi bobot fitur LR, feature importance RF dapat dianalisis	Rendah representasi internal CNN/LSTM sulit diinterpretasikan langsung
Kebutuhan Data	Efektif pada data terbatas	Umumnya membutuhkan data lebih besar
Feature Engineering	Dibutuhkan secara manual (HOG, TF-IDF)	Dapat belajar fitur otomatis dari data
Risiko Overfitting	Relatif rendah pada data kecil	Tinggi jika data terbatas dan tuning kurang optimal
Kebutuhan Hardware	CPU biasa sudah cukup	GPU sangat direkomendasikan untuk training yang efisien

Ringkasan Perbandingan Lintas Kasus:

Kasus	Tipe Data	Model Terbaik	Metode	Kesimpulan
Titanic	Tabular kecil	Random Forest (Acc: 0.8324)	Konvensional	Data kecil + fitur tabular → model konvensional lebih stabil dan efisien
Digit Recognizer	Citra	Deep CNN (Acc: 0.9943)	Deep Learning	Struktur spasial gambar → CNN jauh lebih unggul secara akurasi
Disaster Tweets	Teks pendek	TF-IDF + LR (F1: 0.7723)	Konvensional	Teks pendek + data terbatas → TF-IDF tetap representasi terkuat

Temuan ini menegaskan bahwa pemilihan model harus selalu disesuaikan dengan karakteristik data, bukan sekadar mengikuti tren kompleksitas arsitektur. "More complex is not always better" adalah prinsip yang terbukti pada Kasus 1 dan Kasus 3.

BAB IV KESIMPULAN DAN REKOMENDASI

IV.1. Kesimpulan

Berdasarkan eksperimen pada tiga kasus dengan tipe data berbeda, dapat disimpulkan bahwa deep learning tidak selalu merupakan pendekatan yang paling optimal untuk semua skenario. Performa model sangat dipengaruhi oleh karakteristik data, volume dataset, kompleksitas pola yang harus dipelajari, serta ketersediaan sumber daya komputasi.

Pada Kasus 1 (Titanic), Random Forest menjadi model terbaik dengan akurasi 83.24%, mengungguli MLP yang hanya mencapai 81.01%. Hal ini membuktikan bahwa pada dataset tabular berukuran kecil (~891 baris), metode konvensional lebih stabil, efisien, dan tidak berisiko overfitting dibanding deep learning.

Pada Kasus 2 (Digit Recognizer), Deep CNN memberikan peningkatan performa yang signifikan dengan akurasi 99.43%, mengungguli HOG + SVM (98.37%) dan mengurangi jumlah kesalahan hingga 65.2% (dari 138 menjadi 48 kesalahan). CNN terbukti sebagai arsitektur yang paling sesuai untuk klasifikasi citra karena kemampuannya mempertahankan dan mengeksploitasi struktur spasial secara otomatis.

Pada Kasus 3 (Disaster Tweets), metode konvensional TF-IDF + Logistic Regression kembali unggul dengan F1-score 0.7723, sementara LSTM gagal total dengan F1-score 0.0000 karena cenderung memprediksi semua data sebagai kelas mayoritas. Teks tweet yang pendek, noisy, dan mengandung ambiguitas semantik tidak memberikan kondisi yang mendukung LSTM tanpa pretrained embedding dan data yang lebih besar.

IV.2. Rekomendasi

Berdasarkan temuan eksperimen, rekomendasi praktis yang dirumuskan adalah sebagai berikut:

Untuk dataset tabular berukuran kecil seperti Titanic, gunakan model konvensional seperti Random Forest, XGBoost, atau Gradient Boosting sebagai baseline utama. Optimalkan feature engineering terlebih dahulu. Deep learning baru dipertimbangkan jika data training minimal berada di atas puluhan ribu sampel.

Untuk data citra**,** CNN sangat direkomendasikan. Mulai dengan arsitektur sederhana (Conv-Pool-Flatten-Dense) dan tingkatkan kompleksitas secara bertahap. Gunakan Batch Normalization dan Dropout untuk regularisasi. Pertimbangkan data augmentation untuk meningkatkan generalisasi.

Untuk data teks pendek seperti tweet, TF-IDF + Logistic Regression atau SVM tetap menjadi baseline yang kuat dan sulit dikalahkan tanpa pretrained language model. Jika ingin menggunakan deep learning pada teks, gunakan pretrained embedding (GloVe, FastText) atau fine-tuning Transformer kecil (DistilBERT) daripada melatih embedding dari awal. Terapkan class weighting untuk menangani ketidakseimbangan kelas.

DAFTAR PUSTAKA

Alzubaidi, L., Zhang, J., Humaidi, A. J., Al-Dujaili, A., Duan, Y., Al-Shamma, O., Santamaría, J., Fadhel, M. A., Al-Amidie, M., & Farhan, L. (2021). Review of deep learning: concepts, CNN architectures, challenges, applications, future directions. *Journal of Big Data*, 8(1). https://doi.org/10.1186/s40537-021-00444-8

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. https://www.bioinf.jku.at/publications/older/2604.pdf

Kaggle. (n.d.-a). *Titanic: Machine Learning from Disaster*. Retrieved April 2026, from https://www.kaggle.com/c/titanic

Kaggle. (n.d.-b). *Digit Recognizer*. Retrieved April 2026, from https://www.kaggle.com/c/digit-recognizer

Kaggle. (n.d.-c). *Natural Language Processing with Disaster Tweets*. Retrieved April 2026, from https://www.kaggle.com/c/nlp-getting-started

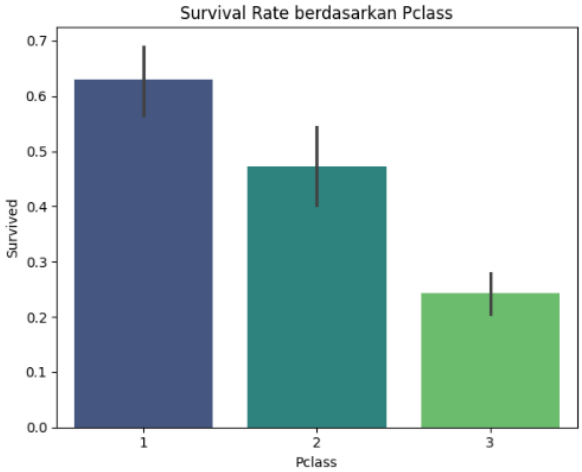
Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830. http://scikit-learn.sourceforge.net

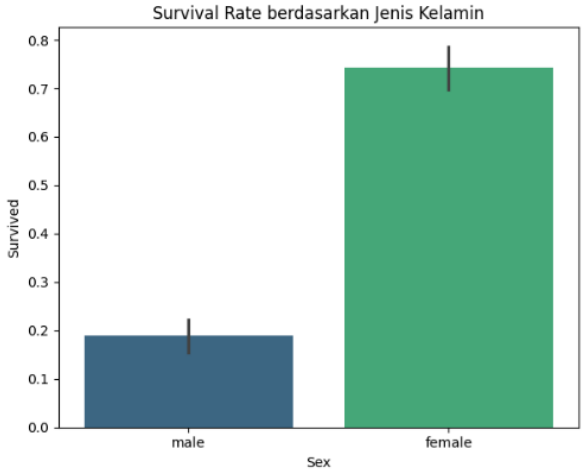
Stanford CS231. (2026). CS231n: Deep Learning for Computer Vision - Convolutional Neural Networks. https://cs231n.github.io/convolutional-networks/

TensorFlow Core. (n.d.). *Keras: High-level API for TensorFlow*. Retrieved April 2026, from https://www.tensorflow.org/guide/keras

LAMPIRAN

Lampiran A — Kasus 1: Titanic

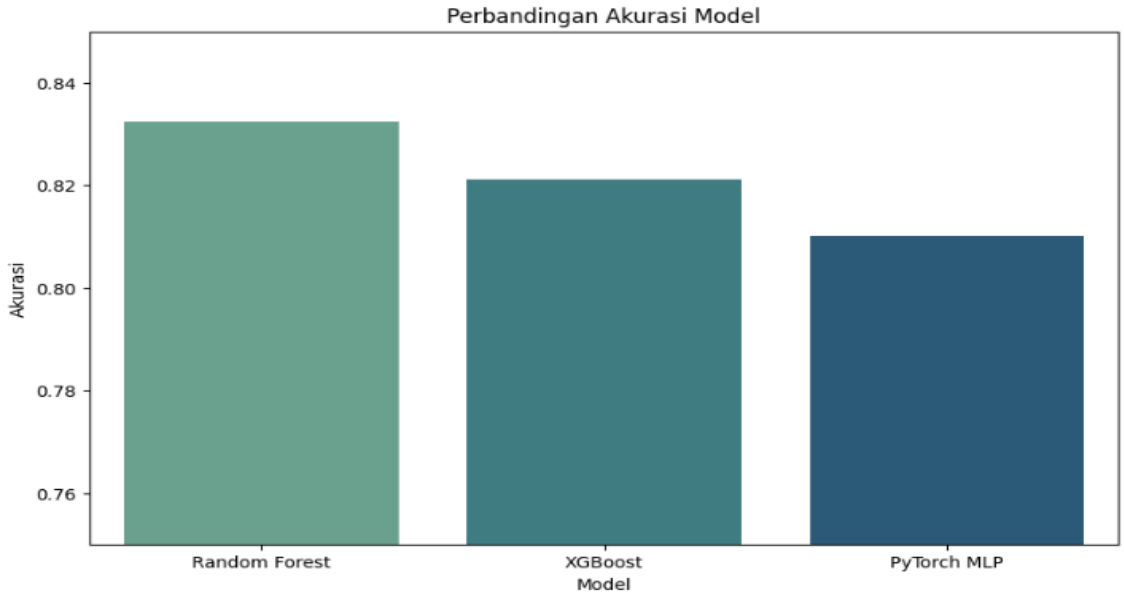




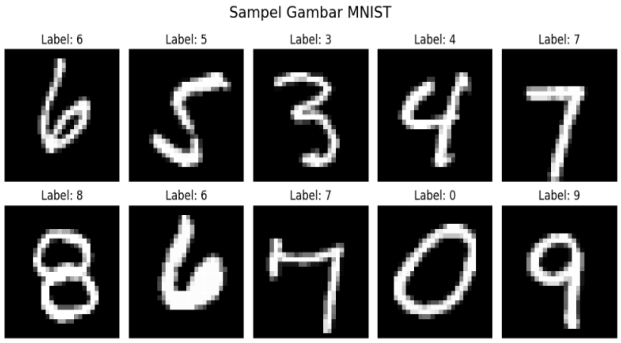

```
[Random Forest] Accuracy: 0.8324
              precision    recall  f1-score   support

           0       0.84      0.88      0.86       105
           1       0.81      0.77      0.79        74

    accuracy                           0.83       179
   macro avg       0.83      0.82      0.83       179
weighted avg       0.83      0.83      0.83       179
```

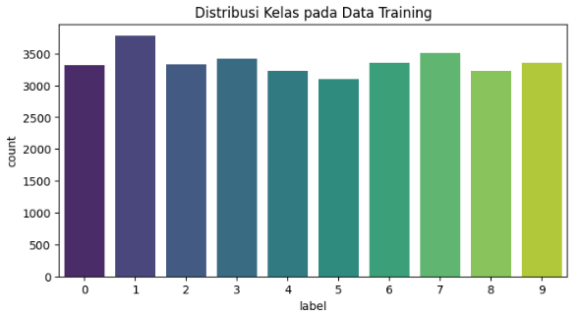
```
Training PyTorch MLP...
Early stopping at epoch 33

[PyTorch MLP] Accuracy: 0.8101
              precision    recall  f1-score   support

           0       0.81      0.89      0.85       105
           1       0.81      0.70      0.75        74

    accuracy                           0.81       179
   macro avg       0.81      0.79      0.80       179
weighted avg       0.81      0.81      0.81       179
```

```
[XGBoost] Accuracy: 0.8212
              precision    recall  f1-score   support

           0       0.83      0.88      0.85       105
           1       0.81      0.74      0.77        74

    accuracy                           0.82       179
   macro avg       0.82      0.81      0.81       179
weighted avg       0.82      0.82      0.82       179
```

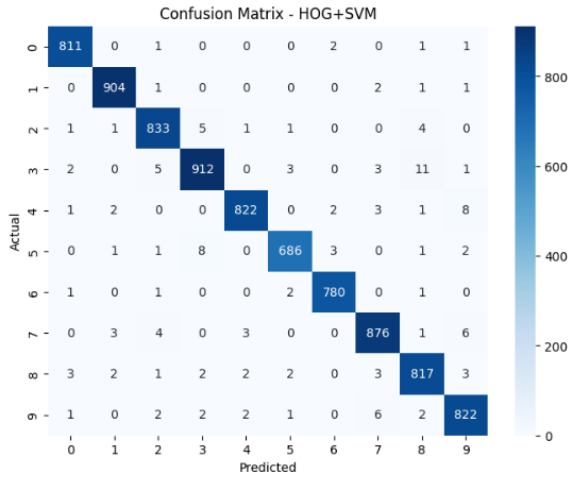


Lampiran B Kasus 2: Digit Recognizer





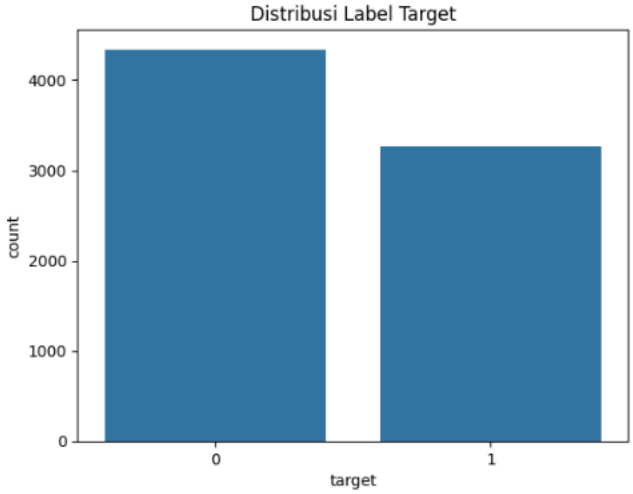

```
--- HOG + SVM Performance ---
Accuracy: 0.9837

Classification Report:
              precision    recall  f1-score   support

           0       0.99      0.99      0.99       816
           1       0.99      0.99      0.99       909
           2       0.98      0.98      0.98       846
           3       0.98      0.97      0.98       937
           4       0.99      0.98      0.99       839
           5       0.99      0.98      0.98       702
           6       0.99      0.99      0.99       785
           7       0.98      0.98      0.98       893
           8       0.97      0.98      0.98       835
           9       0.97      0.98      0.98       838

    accuracy                           0.98      8400
   macro avg       0.98      0.98      0.98      8400
weighted avg       0.98      0.98      0.98      8400
```

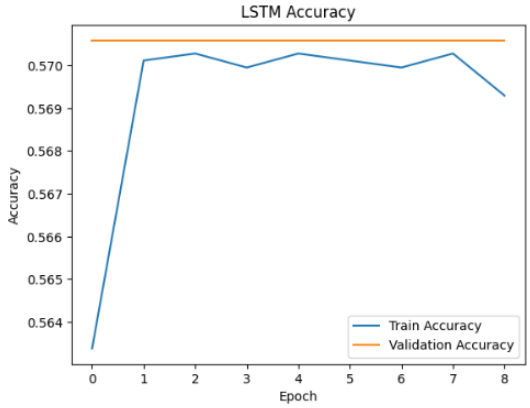


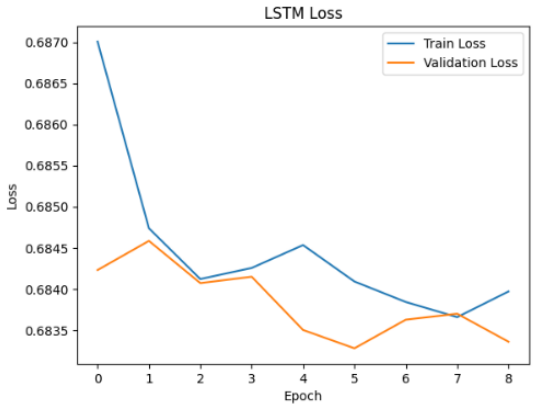

```
Epoch 12/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 113s 216ms/step - accuracy: 0.9920 - loss: 0.0268 - val_accuracy: 0.9925 - val_loss: 0.0221
Epoch 13/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 141s 214ms/step - accuracy: 0.9928 - loss: 0.0226 - val_accuracy: 0.9940 - val_loss: 0.0219
Epoch 14/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 134s 199ms/step - accuracy: 0.9929 - loss: 0.0218 - val_accuracy: 0.9933 - val_loss: 0.0255
Epoch 15/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 104s 198ms/step - accuracy: 0.9936 - loss: 0.0197 - val_accuracy: 0.9923 - val_loss: 0.0215
Epoch 16/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 148s 209ms/step - accuracy: 0.9935 - loss: 0.0205 - val_accuracy: 0.9933 - val_loss: 0.0251
Epoch 17/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 109s 207ms/step - accuracy: 0.9946 - loss: 0.0159 - val_accuracy: 0.9929 - val_loss: 0.0222
Epoch 18/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 112s 213ms/step - accuracy: 0.9949 - loss: 0.0183 - val_accuracy: 0.9939 - val_loss: 0.0214
Epoch 19/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 145s 220ms/step - accuracy: 0.9952 - loss: 0.0142 - val_accuracy: 0.9943 - val_loss: 0.0206
Epoch 20/20
525/525 ━━━━━━━━━━━━━━━━━━━━ 112s 214ms/step - accuracy: 0.9954 - loss: 0.0142 - val_accuracy: 0.9942 - val_loss: 0.0225

--- Deep CNN Performance ---
Validation Accuracy: 0.9943
```

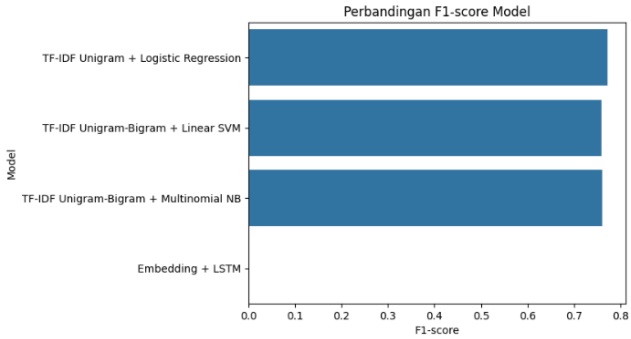
Lampiran C — Kasus 3: Disaster Tweets

Kata yang Sering Muncul pada Tweet Bencana









SUBMISSION KASUS 1

5054	Muhammad Yoga Isnaeni		0.77751	1	30s

Your First Entry!
Welcome to the leaderboard! Your score represents your submission's accuracy. For example, a score of 0.7 in this competition indicates you predicted Titanic survival correctly for 70% of people.

What next? You've got a few options:

- Learn skills that can improve your score in our Intro to Machine Learning course by Dan Becker.
- Check out the discussion forum to find lots of tutorials and insights from other competitors.
- Find a new challenge by entering one of our open, active competitions or searching our public datasets.

SUBMISSION KASUS 2

497	Muhammad Yoga Isnaeni		0.99150	1	8s

Your First Entry!
Welcome to the leaderboard!

SUBMISSION KASUS 3

390	Muhammad Yoga Isnaeni		0.79558	1	6s

Your First Entry!
Welcome to the leaderboard!